

2.3 — Distribute Tools Appropriately Across Agents and Configure Tool Choice

Concept

Every tool you give an agent expands what it can do — and what it can do wrong. **Scoped tool access** (the principle of least privilege applied to agents) means each agent receives only the tools it needs to complete its specific responsibility, and nothing more. A research subagent that only needs to read documents should never hold a `delete_record` or `send_email` tool. If it does, a misinterpreted instruction or prompt injection can trigger side effects that are impossible to roll back.

The pattern is: define tools globally in your application, then pass a filtered subset when constructing each subagent's `tools` array. The model has no awareness of tools that are not listed in its request — it cannot be tricked into calling them. This is the only reliable enforcement boundary.

tool_choice controls how aggressively Claude selects tools in a given turn. There are three modes:

- `{ type: "auto" }` — Claude decides whether to call a tool or respond with text. This is the default and the right choice for most conversational agents. The model uses tools when it judges them necessary.
- `{ type: "any" }` — Claude must call at least one tool from the list, but it chooses which one. Use this when you know a tool call is always required in a particular step — for example, a logging step where every response must be recorded before returning.
- `{ type: "tool", name: "..."` — Claude must call exactly the named tool. This forces deterministic behaviour on a known first step, such as always running a `validate_input` tool before any other action, or always calling a `plan_subtasks` tool before the coordinator begins delegating.

Forced tool selection (`type: "tool"`) is most valuable at the **start of a workflow** where you need a guaranteed structured output before the rest of the system can proceed. After that step resolves, switch back to `auto` for the remaining turns so the agent can reason freely.

In the Agent SDK, `allowedTools` in subagent configuration serves the same scoping purpose at the SDK abstraction layer. The underlying mechanism is identical — only tools in the list are included in the API request — but `allowedTools` makes the intent explicit and keeps tool filtering co-located with subagent definitions rather than scattered across call sites.

Why giving every agent every tool causes confusion: Claude reasons about which tool to call based on the tool descriptions visible in the request. When a focused subagent sees twenty tools it will never need, two problems arise. First, the model spends inference budget disambiguating between irrelevant options, which degrades response quality. Second, with a long tools list, the model is more likely to call a plausible-looking but wrong tool — especially under ambiguous instructions. Narrow tool sets produce sharper, more predictable behaviour.

Code Example

```
import Anthropic from "@anthropic-ai/sdk";

const client = new Anthropic();

// All tools defined once – subsets are distributed per agent
const ALL_TOOLS: Anthropic.Tool[] = [
  {
    name: "validate_input",
    description: "Validate that the user request is well-formed before processing.",
    input_schema: {
      type: "object" as const,
      properties: { input: { type: "string" } },
      required: ["input"],
    },
  },
  {
    name: "search_knowledge_base",
    description: "Search internal documentation for relevant content.",
    input_schema: {
      type: "object" as const,
      properties: { query: { type: "string" } },
      required: ["query"],
    },
  },
  {
    name: "write_file",
    description: "Write content to a file on disk.",
    input_schema: {
      type: "object" as const,
      properties: {
        path: { type: "string" },
        content: { type: "string" },
      },
      required: ["path", "content"],
    },
  },
];

// Forced tool selection – coordinator must call validate_input first
async function coordinatorFirstTurn(userInput: string): Promise<Anthropic.Message> {
  return client.messages.create({
    model: "claude-opus-4-5",
    max_tokens: 1024,
    system: "You are a coordinator. Always validate input before proceeding.",
  });
}
```

```

tools: ALL_TOOLS.filter((t) => t.name === "validate_input"),
// Force the exact tool – no ambiguity, no skipping
tool_choice: { type: "tool", name: "validate_input" },
messages: [{ role: "user", content: userInput }],
});
}

// Research subagent – read-only tools only, auto choice within that set
async function researchSubagent(query: string): Promise<string> {
  const response = await client.messages.create({
    model: "claude-haiku-4-5",
    max_tokens: 1024,
    system: "You are a research specialist. Search the knowledge base and summarize findings.",
    // Scoped: only the search tool – write_file is intentionally excluded
    tools: ALL_TOOLS.filter((t) => t.name === "search_knowledge_base"),
    tool_choice: { type: "auto" }, // Claude decides when to search
    messages: [{ role: "user", content: query }],
  });

  const textBlock = response.content.find((b) => b.type === "text");
  return textBlock?.text ?? "";
}

// Writer subagent – write-only tools, must call at least one tool per turn
async function writerSubagent(content: string, outputPath: string): Promise<void> {
  await client.messages.create({
    model: "claude-haiku-4-5",
    max_tokens: 512,
    system: "You are a file writer. Persist the provided content to the specified path.",
    // Scoped: only the write tool – search is intentionally excluded
    tools: ALL_TOOLS.filter((t) => t.name === "write_file"),
    // "any" ensures a tool is always called – pure text responses are invalid here
    tool_choice: { type: "any" },
    messages: [
      {
        role: "user",
        content: `Write this content to ${outputPath}:\n\n${content}`,
      },
    ],
  });
}

```

Common Mistakes

1. **Passing the full tool list to every agent** — The most common scoping mistake. When all agents share all tools, a research agent can accidentally trigger write operations, and debugging becomes a trace through

tool calls that should never have been reachable. Define tool subsets per agent role and enforce them at the call site.

2. **Using `tool_choice: { type: "tool", name: "..."} for more than one turn`** — Forcing a specific tool is correct for a deterministic first step, but leaving it in place for subsequent turns prevents the model from adapting. After the forced step resolves, switch to `auto` so Claude can reason freely for the remainder of the agentic loop.
3. **Confusing `type: "any"` with `type: "auto"`** — `auto` means "use a tool if you judge it necessary"; `any` means "you must use a tool — pick one." Using `auto` when you need a guaranteed tool call (for example, a logging step) silently allows the model to skip it. Use `any` when a tool call is a hard contract of that turn.

References

- [Anthropic: Tool use — controlling Claude's output with `tool\_choice`](#)
- [Anthropic: Build multi-agent systems — tool access and scoping](#)
- [Anthropic: Agent SDK — allowedTools configuration](#)